

CS160 - Introduction to Computer Science [Course Project Report]

Project Title: Go Game - A Complete Go Application with SFML
Final Version

Nguyễn Phú Bảo Khang - 25125054

Nguyễn Phúc Khánh - 25125086

Date of Submission: December 14th, 2025

CS160 - Introduction to Computer Science [Course Project Report]

Project Title: Go Game - A Complete Go Application with SFML

Team Members:

- Nguyễn Phú Bảo Khang - 25125054
- Nguyễn Phúc Khánh - 25125086

Date of Submission: December 14th, 2025

Version: 02 (Final)

Project Overview:

This project implements a fully-functional Go game in C++, featuring both console-based and graphical interfaces. The primary objective is to design a clean, extensible, and robust architecture that accurately represents the rules, logic, and flow of a standard Go match while providing a smooth user experience.

The application consists of several core components that form a complete Go engine:

- **Board System:** Represents the Go board, intersections, liberties, connected groups, capturing logic, and territory calculation using DFS algorithms. (Implementation: Board.h, Board.cpp)
- **Game Controller:** Manages game flow, turn alternation, legality checking, Ko protection, pass rules, undo/redo stack, move history, komi, endgame scoring, and resignation. (Implementation: Game.h, Game.cpp)
- **Artificial Intelligence (AI):** Provides Easy/Medium/Hard difficulty levels and chooses moves based on heuristics and randomness. (Implementation: AI.h, AI.cpp)
- **Graphical User Interface (GUI):** Built with SFML, featuring interactive board rendering, animations, themes, music player, save/load slots, modal dialogs, and dynamic window resizing with letterboxing. (Implementation: UI.h, UI.cpp)
- **Console Interface:** A full text-based version supporting commands, ASCII rendering, and basic interactions. (Implementation: UI.cpp)

The goals of this project are:

1. Implement the core mechanics of Go accurately, including stone placement, liberty evaluation, capturing, passing, Ko rule, and scoring.

2. Provide two fully operational user interfaces - console and graphical - allowing users to play the game seamlessly.
3. Apply software engineering principles including modularity, separation of concerns, serialization, event-driven design, and consistent memory management.
4. Demonstrate real-world programming skills such as handling user input, file I/O, building complex UI systems, asynchronous operations (AI), and integrating multimedia assets.
5. Create a polished and user-friendly Go experience suitable for course demonstrations and future extensions.

Requirements Specification

REQUIREMENTS SPECIFICATIONS:

1. Building and running the program:

- a. To build the program, open the Terminal in the project root directory. You can use the command

```
cd path/to/directory
```

to navigate to the project root directory.

● **macOS Installation:**

- Install XCode Command Line Tools (Make sure Clang supports C++17 or above):

```
xcode-select --install
```

- Install Homebrew:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- Install CMake: (Ensure that you have installed CMake version 3.16 or later.)

```
brew install cmake
```

● **Windows Installation:**

- *Install CMake:*
 - + Download the *Windows x64 Installer (.msi)* from the [official CMake download page](#).
 - + Run the installer.
 - + **NOTE:** During installation, select *"Add CMake to the system PATH for all users"*.
- *Install the Compiler (MinGW-w64):* The easiest way to get a modern GCC compiler without a complex setup is to use a standalone build like *WinLibs*.
 - + Go to [winlibs.com](#) and download the Zip archive for the latest *GCC version (Win64)*.
 - + Look for: *"GCC x.x.x + LLVM/Clang/LLD/LLDB + MinGW-w64 UCRT"*.
 - + Extract the zip file to a permanent location (e.g., C:\mingw64). Avoid folders with spaces in the name.
- *Add compiler to PATH:*

Press the *Windows Key* and type "*env*", then press Enter to open "*Edit the system environment variables*".

- + Click the *Environment Variables...* button.
- + In the *System variables* (bottom box), find the variable named *Path* and verify it is selected, then click *Edit*.
- + Click *New* and paste the path to your compiler's bin folder.
 - Example: C:\mingw64\bin
- + Click *OK* on all windows to close them.
- **Building the project:** Type the following into the Terminal.
- macOS:

```
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Release -S .. -B .
cmake --build .
```

- Windows:

```
mkdir build
cd build
cmake -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release -S .. -B .
cmake --build .
```

- b. To run the game, open the Terminal in the project root directory. Type the following into the Terminal:

```
# On Linux/macOS:
./build/GoGame
# On Windows:
.\build\GoGame.exe
```

2. *Functional Requirements*

- **Game modes:** 2 players and PVE (AI opponent).
 - *2 players:*
 - + Two players can play against each other on the same device.
 - + The game should manage turns automatically, alternating between Black and White players.
 - *AI opponent:*
 - + Easy mode: The AI makes quick, random moves without much evaluation.
 - + Medium mode: The AI uses the Minimax algorithm with a shallow depth, making reasonable moves with occasional mistakes.

- + Hard mode: The AI uses Minimax with Alpha-Beta pruning, evaluating a deeper move tree to play strategically and reduce mistakes.
- + The AI makes moves efficiently and within a reasonable time (at most 30-60 seconds) for all difficulty levels.
- **User-friendly user interface (UI) that includes:**
 - + A board on which stones are placed.
 - + Turn indicator HUD.
 - + Notifications for recently placed stones and end-game scoring.
 - + Buttons to: Start new game, reset current game, undo last move, redo last undone move, save current game state, load saved game, game customization (*board color, enable/disable sound effects/music, change background music, volume control*).
- **Game logic:**
 - + Only allow the user to do valid moves.
 - + Automatically detect and end the game.
 - + Implement placing stones, capturing stones, endgame scoring according to the *game rules*.
- **Game rules:** The game follows the [Japanese ruleset](#). Specifically:
 - + Prohibition of suicide: Disallow playing moves such that one's own stones would be captured.
 - + Ko rule: One may not play in such a way as to recreate the board position following one's previous move. (From [Wikipedia](#))
 - + **Does not enforce the superko rule.**
 - + Definition of *territory*: an empty intersection is said to belong to a player's *territory* if all stones adjacent to it or to an empty intersection connected to it are of that player's color. (From [Wikipedia](#))
 - + Territory scoring: A player's score is the sum of the number of empty intersections in their *territory* and the number of opponent's stones *captured*. **(Group tax and seki are ignored)**.
 - + Komi: White player has 6.5 extra points as compensation for playing second.
- **Game State Management:**
 - + Handle turn management, ensuring proper transitions between the player's turn and the AI's turn or between two human players.
 - + Store a history stack of placing stones to facilitate undo operations.

- + Use a second stack to facilitate redoing moves.
- + The user can reset the entire game at any time.
- **Save and Load Functionality:**
 - + The user can save the current game state to a file. The program must save the positions of all stones, the current player's turn, and any other relevant game state information.
 - + The user can load a previously saved game: read from the saved file, restore the game state to the last saved point, and allow the players to continue playing from that state.

3. Non-Functional Requirements

- **Cross-platform compatibility:** The user can follow the compilation tutorial, build the game, and run the game on Windows and macOS.
- **Asset management:** The game should load fonts, audio files, and any other relevant assets successfully.
- **Code quality:** The codebase must follow standard C++ naming conventions and be documented.

Implementation Details

Implementation Details:

1. File Structure Diagram

```
CS160-GoGame/  
├── external/  
│   └── SFML/  
├── include/  
│   ├── Board.h  
│   ├── Game.h  
│   ├── AI.h  
│   └── UI.h  
├── src/  
│   ├── main.cpp  
│   ├── Board.cpp  
│   ├── Game.cpp  
│   ├── AI.cpp  
│   └── UI.cpp  
├── assets/  
│   ├── fonts/  
│   ├── music/  
│   └── sounds/  
├── CMakeLists.txt  
└── README.md
```

2. Key components

- SFML library: (external/SFML/)
 - The folder is the source code of the SFML library (version 3.0.2), which is cloned from SFML's GitHub repository.
 - This library is used to display the GUI window, handle the game loop, and load and use fonts and sound files in the program.
- assets folder:
 - Contain fonts, sound files for the program to load and display/play.

3. Codebase:

- CMakeLists.txt:
 - CMakeLists.txt contains instructions for CMake to build the project into an executable file.
- main.cpp is the main program. It includes UI.h.
- The UI.cpp contains the UI class, responsible for running the game loop, rendering SFML graphical interface and console interface, playing sounds, etc.

- The Board.cpp contains the Board class, responsible for stone positions, liberties, capturing, territory logic, and serializing board state into a string and deserializing.
- The Game.cpp contains the Game class, responsible for resolving the game rules, game mechanics (placing stone, passing, resigning, etc.), turn flow, undo/redo, serializing game state into a string, and deserializing.
- The AI.cpp contains the GamePosition class, which is used to represent a game state in the game tree, and the AI class, which uses the Minimax algorithm with Alpha-Beta pruning and a hand-crafted heuristic function to play a reasonable move.

Design Document

Design Document:

1. Introduction:

The Go Game project is built with a modular, extensible architecture designed to separate core game logic, board representation, AI decision-making, and user interface layers. The primary goal of the design is to ensure clean abstraction boundaries, maximize reusability, and support both console and graphical gameplay modes without rewriting core logic.

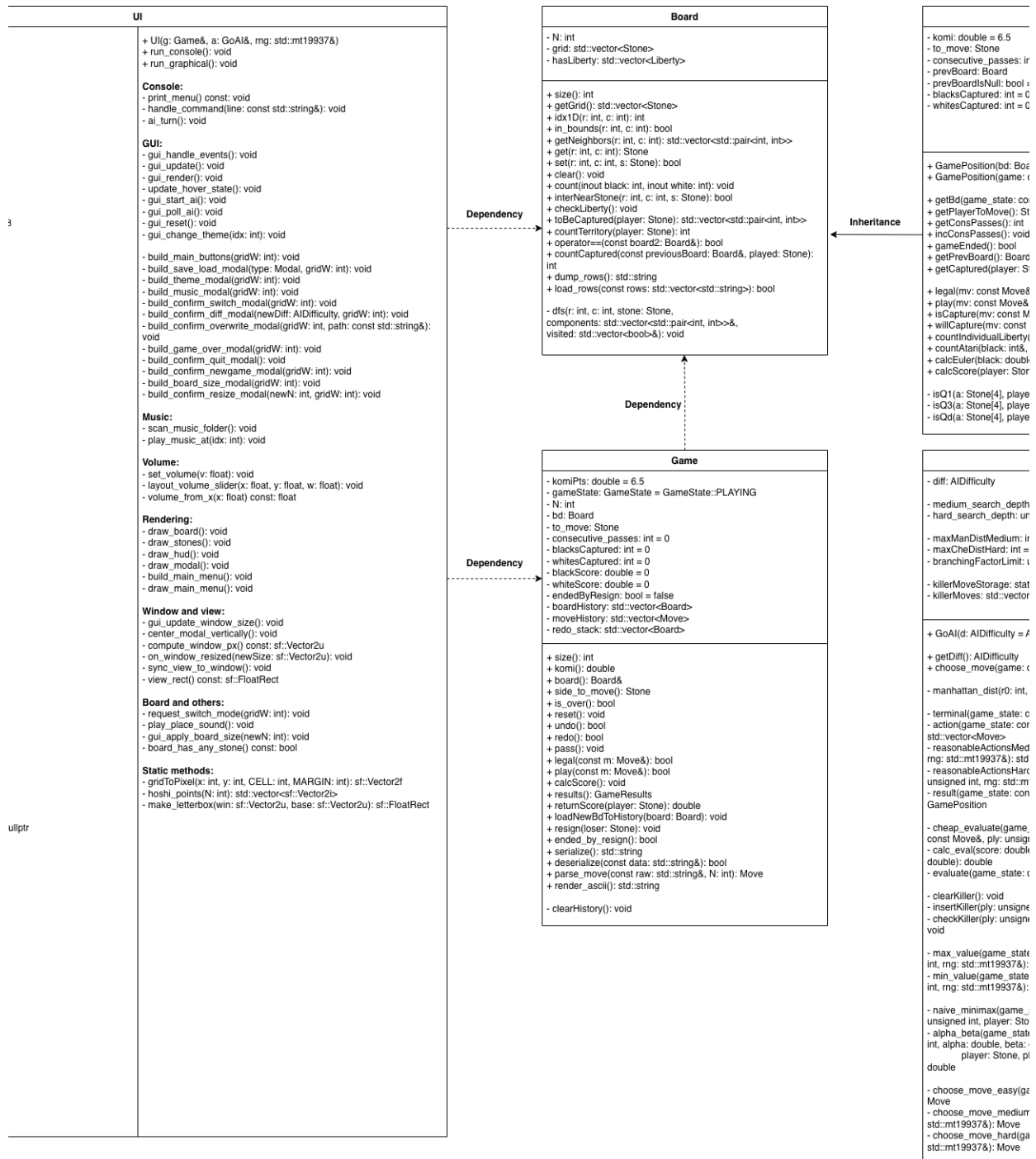
The system consists of four key modules:

- **Board Module:** stone storage, liberties, capturing, and territory logic.
- **Game Module:** rules engine, turn flow, undo/redo, serialization.
- **AI Module:** heuristic-based CPU opponent using minimax and alpha-beta pruning.
- **UI Module:** SFML graphical interface and console interface.

All modules interact in a unidirectional manner: UI → Game → Board.

No UI logic resides in core modules, ensuring clear separation of concerns.

2. High-level Architecture with UML Class Diagram: (Included as GoGameUML.png)



3. Extensibility Considerations:

The system is designed to support:

- Different board sizes (9x9, 13x13, 19x19).
- Board themes (themes vector supports unlimited entries, but only 5 themes were made).

UI detailed explanation

Files: UI.cpp / UI.h

1. Function of UI module:

UI is used for making console interfaces and graphical interfaces.

It will control many properties:

- Event loop, rendering the board, stones, and Heads-Up Display (HUD).
- Control buttons (Undo/ Redo, Pass, Mode, New Game, Save/ Load, Theme, Music, Quit).
- Modal popups (confirmation dialogs, board size selection, theme selection, music selection, etc.).
- Background music, stone placement sound effects, and volume slider.
- Asynchronous AI calls (std::async) and handling the AI's move.

2. Namespaces and File-level constants and helpers:

2.1) Namespaces and aliases:

- `using namespace std::chrono_literals;`
Allows the use of time literals such as: 0ms, 10ms, etc.
- `using namespace fs = std::filesystem;`
Short alias for `std::filesystem`, used for file/path operations (save games, music files, etc.).

2.2) Global constants:

- `MIN_RESIZE_W, MIN_RESIZE_H;`
Minimum allowed window size when the user resizes the window.
- `PANEL_W, PANEL_GAP, RIGHT_PAD;`
Define the layout of the right-side control panel: width, spacing from the board, and padding.
- `MIN_WIN_H;`
Minimum window height.

2.3) Anonymous namespaces (save helpers):

- ```
namespace {
 bool file_noempty(const std::string& p);
 bool file_noempty_abs(const fs::path& p);
 const fs::path& save_root();
 fs::path slot_path(int idx);
}
```

→ `file_noempty, file_noempty_abs`: check whether a file exists, is a regular file and has size > 0.

→ `save_root()`: returns the absolute path to the saves directory (auto-created if missing).

→ `slot_path(int idx)`: maps slots numbers (slot1, slot2, etc.) to paths like 'saves/slotX.sav'

**NOTE:** This kind of namespace is internal to `UI.cpp` only, not exposed outside.

### 3. *Enum classes:*

#### **Enum UIType:**

- `enum class UIType { CONSOLE, GRAPHICAL };`  
Indicates which interface mode the program can run in.  
(It is useful for the overall project structure.)
- `struct Theme {`  
    `sf::Color board, border, grid, star;`  
    `float borderThickness, gridThickness;`  
`};`

Encapsulates all visual attributes of the board:

- Board background color, border color, grid line color, hoshi color.
- Border thickness and grid line thickness.

It allows switching themes simply by assigning a different **Theme**.

# Game detailed explanation

## ***Files: Game.cpp / Game.h***

### **1. Function of Game class:**

The Game class is the central controller of a Go match. It:

- Owns a Board object (bd) and the game configuration (board size, komi).
- Tracks:
  - Which player is to move (to\_move).
  - How many consecutive passes have occurred.
  - How many stones each player has captured.
  - Final scores and whether the game ended by resignation.
  - Maintains full history of board positions (for undo/redo and Ko rule).
- Implements:
  - Move legality (including suicide and a simple Ko check).
  - Game flow functions: play, pass, undo, redo, resign.
  - Endgame scoring: territory + captures + komi.
  - Serialization/Deserialization for save/load.
  - A simple ASCII renderer for console debugging.

### **2. Supporting Types:**

#### **2.1) Enums**

- `enum class GameMode { PVP, PVE };`

Describes the mode of the game:

- PVP: Player vs Player.
- PVE: Player vs Environment (AI).

- `enum class GameState { PLAYING, ENDED };`

Represents the lifecycle state of the current game:

- PLAYING: Game is ongoing; moves and passes are allowed.
- ENDED: Game has finished (either by two passes in a row or by resignation).

Many methods (e.g., `pass()`, `undo()`, `redo()`, `legal()`) check `gameState` to prevent illegal actions after the game is over.

- `enum class GameResults { BLACK_WINS, WHITE_WINS, DRAW };`

Represents the final outcome of the game based on scores.

#### **2.2) Structs**

- `struct Move { int r = 0, c = 0; bool is_pass = false; };`

Represents a single move in the game:

- `r`, `c`: row and column indexes (starting from top-left) of the intersection where a stone is placed.
- `is_pass`: true if the move is a pass (or “RESIGN” from text input); in this case, `r` and `c` are ignored.

This struct is used both by the `logic` (`play`, `pass`) and by `history` (`moveHistory`).

### 3. Game class

#### 3.1) Private data members

- `double komiPts = 6.5;`  
Japanese komi value: 6.5 (bonus points for White to compensate for Black moving first).
- `GameState gameState = GameState::PLAYING;`  
Current game state (playing or ended).
- `int N;`  
Board size. The board has  $N * N$  intersections (e.g.  $9 \times 9$ ,  $13 \times 13$ ,  $19 \times 19$ ).
- `Board bd;`  
The underlying **Go board** that stores stone placements and implements territory/liberty logic.
- `Stone to_move;`  
The color whose turn it is: `Stone::BLACK` or `Stone::WHITE`.  
Initialized so that Black moves first.
- `int consecutive_passes = 0;`  
Number of **consecutive pass moves**.  
When `consecutive_passes >= 2`, the game is considered over (`is_over()` returns true and `gameState` is set to `ENDED`).
- `int blacksCaptured = 0, whitesCaptured = 0;`  
Capture counters:
  - `blacksCaptured`: number of black stones captured by White.
  - `whitesCaptured`: number of white stones captured by Black.
 These are used in final scoring (territory + captures).
- `double blackScore = 0, whiteScore = 0;`  
Final score for each player:

- `blackScore = blackTerritory + whitesCaptured`
- `whiteScore = whiteTerritory + blacksCaptured + komiPts`
- `bool endedByResign = false;`  
Flag indicating whether the game ended by resignation.
  - If `true`, `calcScore()` is skipped, and scores are forced to 10000 vs. 0 to clearly indicate the winner.
- `std::vector<Board> boardHistory;`  
Stores the **full history of board positions** in order:
  - `boardHistory[0]` is the initial empty board.
  - Each time a move is played (including passing), the new `Board` state is appended.
  - This history is used for undo/redo and for a simple form of Ko checking (comparing previous board states).
- `std::vector<Move> moveHistory;`  
Stores all moves played in order:
  - Used as a log for printing/debugging and for potential extensions and features.
  - Not strictly required for applying undo/redo, since `boardHistory` already stores full states.
- `std::vector<Board> redo_stack;`  
Stack of undone board states:
  - When `undo()` is called, the last board state is moved from `boardHistory` to `redo_stack`.
  - When `redo()` is called, a board state is popped from `redo_stack` and re-applied.
- `void clearHistory();`  
Helper to clear:
  - `boardHistory`
  - `moveHistory`
  - `redo_stack`
 and reset `gameState` to `PLAYING`.  
Used by `reset()` and when loading a new board from a save file.

### 3.2) Public: Constructors and basic queries

- `explicit Game(int n = 19);`  
Creates a new game with an  $n \times n$  board:
  - Sets `N = n`.
  - Constructs `bd(n)` as an empty board.
  - Sets `to_move = Stone::BLACK` (Black moves first).
  - Initializes `boardHistory` with one element: the initial empty Board.
- `int size() const;`  
Returns the board size `N`.  
Used by UI / AI to know whether the game is  $9 \times 9$ ,  $13 \times 13$ , or  $19 \times 19$ .
- `double komi() const;`  
Returns `komiPts` (komi value).
- `Board& board();`  
Returns a **mutable reference** to the internal Board.
  - Used by modules that need to read or modify the board state directly (e.g., UI rendering).
- `const Board& board() const;`  
Returns a **const reference** to the internal Board.
  - Used when only reading the board state is required.
- `Stone side_to_move() const;`  
Returns the stone color (`Stone::BLACK` or `Stone::WHITE`) whose turn it currently is.
- `bool is_over() const;`  
Returns `true` if the game is over due to **two or more consecutive passes**:
  - Implementation: `return consecutive_passes >= 2;`
 Note: `gameState` is also set to `ENDED` once `is_over()` is triggered inside `play()` or `pass()`.

### 3.3) Public: Game flow and actions

- `void reset();`  
Resets the game to the initial state:
  - `gameState = GameState::PLAYING`
  - Clears the board (`bd.clear()`) and sets `to_move = Stone::BLACK`.
  - Clears `boardHistory`, `moveHistory`, and `redo_stack`.

- Resets `consecutive_passes`, capture counters, scores, and `endedByResign`.

After `reset()`, the game behaves as a fresh new match.

- `bool undo();`

Undo the last move, if possible:

- If `gameState == ENDED` or there are no previous boards (`boardHistory.size() <= 1`), returns `false`.
  - Otherwise:
    - Takes the second-to-last board from `boardHistory` (`boardHistory[boardHistory.size() - 2]`).
    - If this board is *equal* to the current `bd` (i.e., the last “move” was a pass), only:
      - Switches `turn` with `to_move = opposite(to_move)`.
      - Decrements `consecutive_passes` (if positive).
    - Otherwise (the last move was a stone placement):
      - Sets `bd` back to the previous board.
      - Resets `consecutive_passes = 0`.
      - Switches `turn` back.
    - Moves the last board state from `boardHistory` to `redo_stack`.
- Returns `true` on success.

- `bool redo();`

Redo a previously undone move:

- If `gameState == ENDED` or `redo_stack` is empty, returns `false`.
  - Otherwise:
    - Pops a board from `redo_stack` into `redoneBoard`.
    - Switches `turn` with `to_move = opposite(to_move)`.
    - Sets `consecutive_passes` to:
      - 1 if `bd == redoneBoard` (meaning this is a pass move).
      - 0 otherwise (stone placement).
    - Appends `redoneBoard` to `boardHistory` and sets `bd = redoneBoard`.
- Returns `true` on success.



- `void pass();`

Performs a pass move:

- If `gameState == ENDED`, it does nothing.
- Pushes the current board (`bd`) to `boardHistory`.
- Appends a pass move `Move{0,0,true}` to `moveHistory`.
- Clears `redo_stack` (since new move invalidates redo).
- Increments `consecutive_passes`.
- Switches `to_move` to the opposite color.

If two consecutive passes occur, `play()` will set `gameState = ENDED`.

- `bool legal(const Move& m) const;`

Checks whether a move is legal in the current state:

- If `gameState == ENDED`, return `false`.
  - If `m.is_pass`, always return `true`.
  - Otherwise:
    - Checks that `(m.r, m.c)` is in bounds:  
`bd.in_bounds(m.r, m.c)`.
    - Checks that the intersection is empty: `bd.get(m.r, m.c) == Stone::EMPTY`.
    - Simulates playing the move on a copy of the board (`futureBoard`):
      - If `futureBoard.set(m.r, m.c, to_move)` return `false`, then the move is suicide, which is illegal.
    - Ko rule (simple form):
      - Compares `futureBoard` with the previous board in `boardHistory` (if it exists).
      - If they are equal, the move recreates a previous board position, which is illegal.
- Returns `true` only if all checks pass.

- `bool play(const Move& m);`

Plays a move if it is legal:

- If `legal(m)` is `false`, returns `false`.
- If `m.is_pass`:
  - Calls `pass()`.

- If `is_over()` (two consecutive passes), sets `gameState = ENDED`.
  - Returns `true`.
- For a normal stone placement:
  - Saves `previousBd = bd`.
  - Places the stone on the board: `bd.set(m.r, m.c, to_move)` (capturing opponent stones as needed).
  - Updates capture counters using `bd.countCaptured(previousBd, to_move)`:
    - If Black played, increments `whitesCaptured`.
    - If White played, increments `blacksCaptured`.
  - Appends the new board to `boardHistory` and the move to `moveHistory`.
  - Clears `redo_stack`.
  - Resets `consecutive_passes = 0`.
  - Switches `to_move` to the opposite color.
- Returns `true` on success.
- `void resign(Stone loser);`  
 Ends the game immediately by resignation:
  - Sets `gameState = GameState::ENDED`.
  - Sets `endedByResign = true`.
  - Assigns extreme scores:
    - If Black resigns: `blackScore = 0, whiteScore = 10000`.
    - If White resigns: `whiteScore = 0, blackScore = 10000`.

This avoids running `calcScore()` and clearly indicates the winner.

- `bool ended_by_resign() const;`  
 Returns `true` if the game ended by resignation (`endedByResign` flag).

### 3.4) Public: Scoring

- `void calcScore();`  
 Computes final scores for both players (only meaningful when the game has ended):
  - If `endedByResign` is `true`, returns immediately (scores already set by `resign()`).

- Otherwise:
  - Calls `bd.countTerritory(Stone::BLACK)` and `bd.countTerritory(Stone::WHITE)`.
  - Prints debug information about territories and captures to `std::cout`.
  - Sets:
    - `blackScore = blackTerritory + whitesCaptured;`
    - `whiteScore = whiteTerritory + blacksCaptured + komiPts;`

These scores are used to determine the winner and are returned by `returnScore()`.

- `GameResults results();`

Returns the final game result:

- Calls `calcScore()` first to ensure scores are up to date.
- Compares `blackScore` and `whiteScore`:
  - `blackScore > whiteScore` means `GameResults::BLACK_WINS`
  - `blackScore < whiteScore` means `GameResults::WHITE_WINS`
  - Otherwise, return `GameResults::DRAW`.

- `double returnScore(Stone player);`

Returns the score of the specified player:

- Calls `calcScore()` first.
- If `player == Stone::BLACK`, returns `blackScore`.
- If `player == Stone::WHITE`, returns `whiteScore`.
- Otherwise, returns `-1` (invalid).

### 3.5) Public: Serialization and loading

- `std::string serialize() const;`

Saves the current game state into a text string suitable for writing to a save file.

*Format:*

- First line: header in the form:  
`GOSAVE N=<N> side=<0|1> komi=<komi>`  
`passes=<passes> blacksCaptured=<...>`

`whitesCaptured=<...>`

Where:

1. `N`: board size.
2. `side`: 0 for Black to move, 1 for White to move.
3. `komi`: komi value (`komiPts`).
4. `passes`: number of consecutive passes.
5. `blacksCaptured`, `whitesCaptured`: capture counts.

- Following lines: board matrix produced by `bd.dump_rows()` (N lines of N characters).

- **`bool deserialize(const std::string& data);`**

Loads a game state from a text string:

- Uses `std::istringstream` to read line by line.
- Strips UTF-8 BOM (Byte Order Mark) if present.
- Has two modes:

1. **Header present** (first line starts with "GOSAVE"):

- Parses fields `N=`, `side=`, `komi=`, `passes=`, `blacksCaptured=`, `whitesCaptured=` using a small `take(key)` lambda.
- Creates a new board `bd` of size `N`, sets `komiPts`, `to_move`, `consecutive_passes`, capture counters.
- Reads exactly `N` rows of board matrix and calls `bd.load_rows(rows)`.
- If loading succeeds, calls `loadNewBdToHistory(bd)` and returns `true`.

2. **Matrix-only** (no header):

- Treats the first line as row 0 of the board.
- Reads `N` lines total via an `only_rows(...)` helper, and calls `bd.load_rows(rows)`.
- On success, calls `loadNewBdToHistory(bd)` and returns `true`.

Returns `false` if the format is invalid or the rows cannot be loaded.

- **`void loadNewBdToHistory(Board board);`**

Used when loading a saved game:

- Calls `clearHistory()` to clear all histories and reset `gameState`.

- Pushes the new `board` into `boardHistory`.  
This makes the loaded board the starting point of the current game.

### 3.6) Public: Console helpers

- `static Move parse_move(const std::string& raw, int N);`

Parses a text command (e.g., "D4", "Q11", "pass", "RESIGN") into a `Move`:

- Trims whitespace using `trim(...)`.
  - Converts letters to uppercase.
  - If the string is "PASS" or "RESIGN", returns a pass move `{0, 0, true}`.
  - If the string is too short (`< 2`), treat it as a pass.
  - Otherwise:
    - Uses `col_from_char(s[0])` to map the letter column to an index (skipping I).
    - Uses `std::stoi(s.substr(1)) - 1` to parse the row index.
    - If indices are out of range, returns a pass move.
    - Else returns `{r, c, false}`.
  - `std::string render_ascii() const;`
- Renders the current board into an ASCII-art string:
- Top and bottom: column labels A..T (skipping I) using `char_from_col(c)`.
  - Each row:
    - Left and right row labels with numbers N..1.
    - Intersection characters: `'.'`, `'X'`, `'O'` via `stone_char(bd.get(r, c))`.
  - Uses `std::ostringstream` to build the string.

This function is mainly for debugging or console mode.

### 3.7) Interaction with Board and utilities

- Game depends heavily on `Board` for low-level rules:
  - `bd.in_bounds`, `bd.get`, `bd.set` for placing stones and checking legality.
  - `bd.countTerritory` for endgame scoring.

- `bd.countCaptured` for counting captures after a move.
  - `bd.dump_rows` and `bd.load_rows` for save/load.
- It also uses utility functions from `Board.h`:
  - `trim` for cleaning user input and header lines.
  - `col_from_char` / `char_from_col` for converting between letter coordinates and column indices.
  - `stone_char` for ASCII rendering.

## Board detailed explanation

**Files: *Board.cpp* / *Board.h***

### **1. Function of Board class:**

The Board class is used for representing the Go board in the game.

The Board class keeps track of:

- Board size (9x9, 13x13, or 19x19), stones' positions
- Which stones have liberties

### **2. Constants and enum classes:**

#### **2.1) Constants:**

```
const std::vector<std::pair<int, int>> offsets = {
{-1, 0}, {1, 0}, {0, -1}, {0, 1} };
```

We add one of these pair values to the 2D coordinates (row, column) of an intersection to get the coordinates of an intersection adjacent to it.

#### **2.2) Enum class:**

- `enum class Stone { EMPTY, BLACK, WHITE };`

Define an enum class Stone to represent the state of an intersection.

- `enum class Liberty { EMPTY, HAS_LIBERTY, NO_LIBERTY };`

Define an enum class Liberty to denote if an intersection has liberty or not (EMPTY means intersection with NO STONES).

### **3. Board class:**

#### **3.1) private:**

- `int N`

The size of the board. (The number of intersections is  $N * N$ ).

- `std::vector<Stone> grid`

The 1D vector represents the state of the board.

The state of the intersection at row  $r$  and column  $c$  is `grid[r * N + c]`.

- `std::vector<Liberty> hasLiberty`

The 1D vector represents whether the stone at an intersection has a liberty or not.

- `void dfs(int r, int c, Stone stone, std::vector<std::pair<int, int>>& components, std::vector<bool>& visited) const;`

This function performs Depth-first Search to find all intersections that are connected to  $(r, c)$  and have state `stone`.

#### **3.2) public: Constructors and basic queries**



- `explicit Board(int n = 19)`

Creates a new, empty  $N * N$  Go board, where  $N$  is the input argument  $n$ .

*Internally:*

- $N$  is set to  $n$ .
  - `grid` is initialized as a 1D vector of size  $N \times N$ , filled with `Stone::EMPTY`.
  - `hasLiberty` is initialized as a 1D vector of size  $N \times N$ , filled with `Liberty::EMPTY`.
- `int size() const`  
Returns the board size  $N$ , which is the number of intersections on one side of the board, for query purposes.
- `std::vector<Stone> getGrid() const`  
Returns a copy of the internal `grid` vector.
- `int idx1D(int r, int c) const`  
Converts 2D coordinates  $(r, c)$  (row, column) into a 1D index in the `grid` vector. The formula is:  $\text{index} = r * N + c$ .

### 3.3) public: Bounds checking and neighborhood

- `bool in_bounds(int r, int c) const`  
Returns `true` if  $(r, c)$  lies inside the board; `false` otherwise.

*Formally:*

- $0 \leq r < N$  and  $0 \leq c < N$  means valid.
  - Anything outside that range is invalid.
- `std::vector<std::pair<int, int>> getNeighbors(int r, int c) const`  
Returns a list of valid neighbors (adjacent intersections) of  $(r, c)$ .

*Implementation:*

- Loops through the constant `offsets = {(-1,0), (1,0), (0,-1), (0,1)}`.
  - For each offset  $(dx, dy)$ , check if  $(r + dx, c + dy)$  is inside the board using `in_bounds`.
  - If valid, pushes  $(r + dx, c + dy)$  into the result vector.

### 3.4) public: Reading and writing intersections

- `Stone get(int r, int c) const`  
Operation to read the board. Returns the state (`Stone::EMPTY`, `Stone::BLACK`, or `Stone::WHITE`) of the intersection at  $(r, c)$ .

*Behavior:*

- Uses `assert(in_bounds(r, c))` to guarantee the coordinates are valid in debug builds.
- `bool set(int r, int c, Stone s)`  
Places a stone `s` at `(r, c)` and applies Go rules related to liberties and captures.  
*Steps:*
  - Assert that `(r, c)` is inside the board.
  - Place the stone: `grid[r * N + c] = s`.
  - Call `toBeCaptured(opposite(s))` to find all opponent stones with no liberties, and remove them (set to `Stone::EMPTY`).
  - Call `toBeCaptured(s)` to find all of **our own** stones with no liberties, and remove them.
  - *Prohibition of suicide:* If any of our own stones are captured in step 4, the move is considered suicide, and the function returns `false`.
  - Otherwise, returns `true`, indicating a legal, non-suicide move.
  - *Note:* The caller (e.g, `Game`) is responsible for checking legality, undoing the move if needed, and updating game state.
- `void clear()`  
Resets the board state to empty.
  - Sets all entries in `grid`, `hasLiberty` to `Stone::EMPTY`, `Liberty::EMPTY` respectively.
- `void count(int& black, int& white) const`  
Counts how many Black and White stones are currently on the board by iterating through `grid`.

### 3.5) public: Liberty and adjacency helpers

- `bool interNearStone(int r, int c, Stone stone) const`  
Checks whether the intersection `(r, c)` is adjacent to at least one neighbor with state `stone`.
- `void checkLiberty()`  
Computes the liberty status for every intersection and writes results into `hasLiberty`.

*Algorithm:*

1. Initialize a visited vector `visited` of size `N * N` with `false`.
2. For each intersection `(r, c)`:
  - If not visited, run `dfs(r, c, componentStone, components, visited)` to find the fully connected

component of intersections that have the same `Stone` state as  $(r, c)$ .

- Check each intersection in components:
  - If any intersection in the component is adjacent to `Stone::EMPTY` (checked via `interNearStone`), the entire component has at least one liberty.

3. For each intersection in components:

- If `componentStone == Stone::EMPTY`, set `hasLiberty[idx] = Liberty::EMPTY` (we don't track liberties for empty intersections).
- Otherwise, set `hasLiberty[idx] = Liberty::HAS_LIBERTY` or `Liberty::NO_LIBERTY` depending on whether the component had a liberty.

This function ***must*** be called before any operation that relies on up-to-date liberty information (e.g., `toBeCaptured`).

- `std::vector<std::pair<int, int>> toBeCaptured(Stone player)`
  - Returns a vector of coordinates  $(r, c)$  of all stones belonging to `player` that currently have **no liberties** and thus will be captured.
  - Implemented by: calling `checkLiberty()` to ensure `hasLiberty` is up to date, then iterating through all of the player's stones and checking the `hasLiberty` vector.

This function is used by `set(...)` to determine which stones need to be removed from the board after a move is played.

### 3.6) public: Territory and scoring helpers

- `int countTerritory(Stone player) const`

Counts how many empty intersections belong to the territory of `player`. Territory is defined as regions of `Stone::EMPTY` intersections that are adjacent **only** to one color of stones (black or white), and not to both.

*Algorithm:*

1. Initialize `territoryCounter = 0` and a `visited` vector for empty intersections.
2. For each intersection  $(r, c)$ :
  - Run `dfs(r, c, Stone::EMPTY, components, visited)` to obtain a connected component of empty intersections.

- For this component, check:
  - `inBlackTerritory`: true if any empty intersection in the component is adjacent to a `Stone::BLACK` stone.
  - `inWhiteTerritory`: true if any empty intersection in the component is adjacent to a `Stone::WHITE` stone.

3. If:

- `player == Stone::BLACK` and `inBlackTerritory == true` and `inWhiteTerritory == false`, then the component is black territory.
- `player == Stone::WHITE` and `inWhiteTerritory == true` and `inBlackTerritory == false`, then the component is white territory.

4. In either valid case, add `components.size()` to `territoryCounter`.

The function returns `territoryCounter`, which is used by `Game` when calculating the final score at the end of a game.

### 3.7) public: board comparison and capture counting

- `bool operator==(const Board& board2) const`

Overloading operator comparing the current board with `board2`. Two boards are equal if and only if every intersection has the same stone state.

- `int countCaptured(const Board& previousBoard, Stone played) const`

Counts how many stones were captured after playing a move.

Implemented by counting the number of Black and White stones on the previous and current boards, then subtracting them.

*Algorithm:*

1. Count the number of black and white stones on the **previous** board.
2. Count the number of black and white stones on the **current** board.
3. If `played == Stone::BLACK`, the number of captured white stones is `prevWhite - curWhite`.
4. If `played == Stone::WHITE`, the number of captured black stones is `prevBlack - curBlack`.

The function returns that number of captured stones.

This is used by the `Game` class to update capture counts and final scores.

### 3.8) public: Serialization helpers

- `std::string dump_rows() const`

Serializes the board into a multi-line string representation suitable for saving to a file.

*String format:*

- There are exactly N lines. Each line has exactly N characters.
- Each character is:
  - 'X' for `Stone::BLACK`
  - 'O' for `Stone::WHITE`
  - '.' for `Stone::EMPTY`

This function is used by `Game::serialize()` when writing the board state to a save file.

- `bool load_rows(const std::vector<std::string>& rows)`

Deserializes the board from a vector of strings (each string is one row).

*Requirements:*

- The vector must contain exactly N strings. Each string must have length exactly N.

*Implementation:*

- For each row `r` and column `c`, reads character `ch = rows[r][c]`.
- Maps:
  - 'X' to `Stone::BLACK`
  - 'O' to `Stone::WHITE`
  - '.' and everything else to `Stone::EMPTY`
- Writes directly into `grid[r * N + c]`.  
On success, return `true`. If the input dimension is inconsistent (wrong number of rows or columns), return `false`.

This function is used by `Game::deserialize()` when loading a saved game.

### 3.9) private: Depth-first search (DFS) helper

- `void dfs(int r, int c, Stone stone, std::vector<std::pair<int, int>>& components, std::vector<bool>& visited) const`

Performs a Depth-first Search starting from `(r, c)` to find a **connected component** of intersections of the same stone type.

*Steps:*

1. Compute `idx = idx1D(r, c)`.

2. If `visited[idx]` is already `true` or `get(r, c) != stone`, return immediately.
3. Otherwise:
  - Mark `visited[idx] = true`.
  - Push `(r, c)` into `components`.
4. For each neighbor `(r0, c0)` of `(r, c)` (using `getNeighbors`):
  - If `get(r0, c0) == stone`, recursively call `dfs(r0, c0, stone, components, visited)`.

*Uses:*

- When `stone` is a **color** (`BLACK` or `WHITE`), `dfs` groups connected stones into one component (a chain or group).
- When `stone == Stone::EMPTY`, `dfs` groups connected empty intersections (an empty region).

# AI detailed explanation

## ***Files: AI.cpp / AI.h***

### **1. Constants, structs, and enum classes:**

The file comes with constants (as `constexpr`) to represent infinity values, the passing move, and the maximum number of plies for the AI to store its killer moves.

The `Branch` struct specifies a branch of the game tree that leads one game state to another. The branch contains information about the move that leads one game state to another, and the evaluation value of that move, to be used for move ordering later.

The `AIDifficulty` enum class is used to specify the difficulty of the AI itself: `EASY`, `MEDIUM`, or `HARD`.

### **2. Function of the `GamePosition` class (Inherited from `Board`):**

*Overview:*

This class encapsulates a snapshot of the game at a specific moment, or a ***game state***. It is responsible for:

- Validating move legality (suicide rules, Ko rule).
- Managing board history (previous states).
- Calculating board metrics (Liberties, Atari states, Territory).
- Pattern recognition (Euler number calculation for eye shapes).

Not only this class contain all information and methods of a `Board` object, but it also contains important data about the game itself: komi, which side is to move, how many consecutive passes there are, how many stones are captured, and the previous board to enforce the Ko rule.

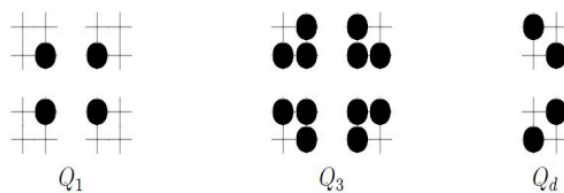
#### ● **Methods:**

- `legal`, `play`: The methods play out just like the `Board` class, but it also updates important game-related information. It tracks the number of captured stones (`blacksCaptured`, `whitesCaptured`), which are crucial for the final score calculation.
- `gameEnded`: The class also tracks `consecutive_passes`. If this counter reaches 2 (both players pass), `gameEnded()` returns true, triggering the terminal state evaluation.
- `isCapture`, `willCapture`: These methods check if a move is a capture move (to exclude them from being a killer move later on), and count how many stones would be captured if that move were played on the board.



To evaluate the board state effectively, the `GamePosition` class computes several sophisticated metrics beyond simple territory counting and scoring:

- **Liberty Counting:** The method `countIndividualLiberty` sums the liberties of all groups. This is critical for tactical safety, where groups with few liberties are vulnerable.
- **Atari Detection:** The `countAtari` method specifically identifies stones with exactly one liberty remaining. This allows the AI to recognize immediate threats or capture opportunities.
- **Euler Number Calculation (`calcEuler`):** A heuristic to connect stones and create eyes separately would be computationally expensive. Luckily, there is a method to do both using the Euler number. Erik van der Werf's thesis paper gives us a method to calculate the Euler number, a topological heuristic that recognizes eyes and connected groups of stones. Erik van der Werf's paper also states that minimizing the Euler number increases the likelihood of connecting stones and creating eyes. The method scans 2x2 windows of the board to identify and count these specific patterns:



The Euler number is then calculated using the following formula:

$$E = \frac{n(Q_1) - n(Q_3) + 2n(Q_d)}{4}$$

### 3. Function of the GoAI class:

*Overview:*

This class acts as the brain of the AI agent. It contains:

- Search algorithms (`naive_minimax`, `alpha_beta`).
- Evaluation functions (`evaluate`, `cheap_evaluate`).
- Heuristic data structures (`killerMoves` vector).
- Move generation logic (`reasonableActionsHard`).

The AI is designed with a modular difficulty system (Easy, Medium, Hard), allowing it to scale its computational intensity and strategic depth:

- **Easy Mode:** Prioritizes speed using randomized move selection among legal candidates.
- **Medium Mode:** Introduces a naive depth-limited Minimax search with basic positional heuristic function.
- **Hard Mode:** Implements a Minimax with Alpha-Beta Pruning search, enhanced by the killer heuristic and move ordering to increase pruning efficiency.
- **Methods:**
  - **terminal:** Checks whether the game state is ended (terminal state).
  - **action:** Returns a RANDOMLY SHUFFLED vector of ALL legal moves possible from that position, including pass.

a) **Easy Mode:**

The AI simply scans all intersections on the board. For each intersection, it considers whether placing a stone on that intersection is a legal move. For all legal moves (including a passing move), it simply picks a random move to be played.

b) **Medium Mode:**

For the Medium Mode, the AI employs a naive standard **Minimax Algorithm** (naive\_minimax).

- **Depth:** Limited to `medium_search_depth` (default: 2).
- **Logic:** The AI recursively alternates between Maximizing (Black) and Minimizing (White) layers. It explores the game tree to find the move that maximizes its gain while assuming the opponent plays optimally.
- **Optimization:** Considering every single move on the board is too costly. The AI uses a basic filter (`reasonableActionsMedium`) to ignore moves that are too far from existing stones (Manhattan distance > 3), reducing the branching factor significantly compared to a full-board search.
- **Terminal state handling:** It relies on the `terminal()` check. If the game ends (double pass) or the depth limit is reached, the search stops and calls the heuristic `evaluate()` function to assign a numerical score to the board position.

c) **Hard Mode:**

The Hard Mode utilizes **Minimax** with **Alpha-Beta Pruning** (`alpha_beta`) to search deeper and more efficiently.

- **Depth:** Extended to `hard_search_depth` (default: 3).

- **Optimization:** Instead of evaluating every leaf node, the algorithm maintains **alpha** (best max found so far) and **beta** (best min found so far) values. Branches that cannot possibly influence the final decision (because a better alternative already exists) are "pruned" or skipped.
- **Efficiency:** This allows the AI to search deeper than standard Minimax while not costing more time exponentially, provided the move ordering is good.
- **Terminal state handling:** It relies on the `terminal()` check. If the game ends (double pass) or the depth limit is reached, the search stops and calls the heuristic `evaluate()` function to assign a numerical score to the board position.

### *More optimization strategies for Hard Mode:*

- **Localized Search (Tunneling)**

A distinct design decision in Hard Mode is the Locality-Based Pruning implemented in `reasonableActionsHard`.

- **Logic:** Instead of scanning the entire board, the AI identifies the `mostRecentMove` made by the opponent.
- **Chebyshev Radius:** It generates candidate moves only within a Chebyshev distance (square radius) of `maxCheDistHard` (default: 3) around the last played stone. In Go, immediate responses are usually local. This heuristic drastically reduces the branching factor, allowing the engine to calculate heuristic values in a specific area of the board without wasting resources on the other side of the board.
- **Trade-off:** While efficient for tactical exchanges, this approach creates a "blind spot" (Tunnel Vision) where the AI may ignore large threats elsewhere on the board if they are far from the current action.

- **Killer Heuristic**

To optimize the Alpha-Beta pruning a bit, the engine implements the **Killer Heuristic**.

- **Concept:** A move that causes a "cutoff" (is so good that the opponent would avoid the branch leading to it) is likely to be good in other similar positions (sibling nodes).
- **Storage:** The class maintains `killerMoves`, a table of size `[_MAX_PLIES_][2]`. It stores up to two "killer" moves for every ply (depth level) of the search.

- **Update Logic:** Inside `alpha_beta`, whenever a beta-cutoff occurs (for MAX) or an alpha-cutoff (for MIN), the move responsible is stored via `insertKiller`.
- **Maintenance:** The function `clearKiller()` is called at the start of every turn to ensure old heuristics do not corrupt the new search.

- **Move Ordering Strategy**

Alpha-Beta pruning is only effective if good moves are searched *first*. The engine implements a rigorous **Move Ordering** phase in `reasonableActionsHard`:

1. **Generation:** Legal moves are generated.
2. **Scoring (`cheap_evaluate`):** Moves are assigned a temporary priority score:
  - **Primary Killer Move:** Score 900.
  - **Secondary Killer Move:** Score 800.
  - **Standard Moves:** Score 0.
3. **Sorting:** The `legalMoves` vector is sorted based on these scores. This ensures that if a "Killer Move" is available in the current position, it is searched immediately. If it triggers a cutoff, the AI saves evaluating dozens or hundreds of other branches.
4. **Branching Factor Control:** To prevent the search tree from exploding, the AI enforces a hard limit on the number of moves checked per node: `branchingFactorLimit` (default: 25). This ensures consistent performance and response times.

- **Evaluation function**

The static evaluation function (`evaluate`) combines various metrics into a single scalar value. The weights used in `calc_eval` reflect the AI's strategic priorities:

- Score (Territory + Captured stones): Weight 1.5. Base importance.
- Liberties Count: Weight 1.0. encourages safety.
- Stone Count: Weight 5.0. Heavily rewards placing stones/staying on board.
- Atari: Weight -20.0. A massive penalty for being in Atari. This makes the AI extremely reactive to saving its own dying stones.
- Euler: Weight -4.0. Penalizes poor shape/inefficient connections.

The weights are hand-picked and tuned after playing several short rounds with the AI, and can be tuned further. The values are multiplied by the weights

and added together. This creates a quick, simple evaluation function that runs in  $O(N)$  time to determine the score (where  $N$  is the number of intersections).

# Game Development Tutorial

# Game Development in C++ (Step-by-Step Tutorial)

Guide to setting up a C++ game with SFML 3.0 and CMake.

## Table of Contents:

1. [Introduction](#)
2. [Environment setup](#)
  - a. [Operating Systems and Tools](#)
  - b. [Installing tools](#)
  - c. [Verifying installation](#)
3. [Project structure & CMake build system](#)
  - a. [Folder layout \(tree\)](#)
  - b. [Writing CMakeLists.txt](#)
  - c. [Useful libraries \(SFML\)](#)
4. [Writing a starter Game/GUI program](#)
  - a. [Writing .cpp files](#)
  - b. [Writing header \(.hpp\) files](#)
  - c. [Adding assets](#)
5. [Building the project](#)
6. [Running the program](#)
7. [Conclusion & Next steps](#)

## 1. Introduction:

- Game development in C++ may seem complex due to compilers, build systems, and graphics libraries (syntax, how to use, etc.).
- However, modern tooling like SFML and CMake has made it much easier to create professional-quality 2D games and graphical applications.
- For robustness and cross-platform compatibility, we will use the terminal combined with an IDE (e.g., Visual Studio on Windows) or code editor (e.g., Sublime Text) in this tutorial.

## 2. Environment Setup:

### a. Operating System and Tools:

#### macOS:

- XCode and Clang C++ compiler
- Homebrew

- CMake
- SFML 3.0.2 library

### **Windows:**

- MinGW-w64 C++ compiler
- CMake
- SFML 3.0.2 library

### **b. Installing Tools:**

#### ***macOS Installation:***

- Install XCode Command Line Tools (Make sure Clang supports C++17 or above):

```
xcode-select --install
```

- Install Homebrew:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- Install CMake: (Ensure that you have installed CMake version 3.16 or later.)

```
brew install cmake
```

#### ***Windows Installation:***

##### ***- Install Visual Studio:***

- + Download the **Visual Studio Installer**.
- + Select the **Desktop development with C++** workload.
- + On the "Installation details" (sidebar), ensure the following are checked:
  - **MSVC v143 - VS 2022 C++ x64/x86 build tools** (The compiler).
  - **C++ CMake tools for Windows** (This integrates CMake into the IDE).
- + Click **Install/Modify**.

##### ***- Install CMake:***

- + Download the *Windows x64 Installer* (.msi) from the [official CMake download page](#).
- + Run the installer.
- + **NOTE:** During installation, select *"Add CMake to the system PATH for all users"*.

- ***Install the Compiler (MinGW-w64):*** The easiest way to get a modern GCC compiler without a complex setup is to use a standalone build like *WinLibs*.



- + Go to [winlibs.com](http://winlibs.com) and download the Zip archive for the latest *GCC version (Win64)*.
- + Look for: "*GCC x.x.x + LLVM/Clang/LLD/LLDB + MinGW-w64 UCRT*".
- + Extract the zip file to a permanent location (e.g., C:\mingw64). Avoid folders with spaces in the name.
- **Add compiler to PATH:** Press the *Windows Key* and type "*env*", then press Enter to open "*Edit the system environment variables*".
  - + Click the *Environment Variables...* button.
  - + In the *System variables* (bottom box), find the variable named *Path* and verify it is selected, then click *Edit*.
  - + Click *New* and paste the path to your compiler's bin folder.
    - Example: C:\mingw64\bin
  - + Click *OK* on all windows to close them.

### c. Verifying Installation:

You can verify the installation by opening a new Terminal window and typing:

#### **macOS Installation:**

```
clang++ --version
cmake --version
```

#### **Windows Installation:**

```
g++ --version
cmake --version
```

## 3. Project Structure & CMake Build System:

CMake allows you to work on macOS, Windows, and Linux with the same code.

### a. Folder Layout (Tree):

(**Note:** Replace *MyProject* with your project name.)

```
MyProject/
├── external/
│ └── SFML/ (after downloading SFML 3.0.2 by following section 3d.)
├── include/
│ └── Game.hpp
├── src/
│ ├── main.cpp
│ └── Game.cpp
├── assets/
└── font.ttf (a font file)
```

## b. Writing CMakeLists.txt:

*(Explanations of CMake commands are also included as comments.)*

```
Require minimum CMake version and create C++ project called MyProject
cmake_minimum_required(VERSION 3.16)
project(MyProject LANGUAGES CXX)

Require modern C++17
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

Point to SFML subdirectory
add_subdirectory(external/SFML)

Add include directory
include_directories(${PROJECT_SOURCE_DIR}/include)

Add your source files
file(GLOB_RECURSE SOURCES CONFIGURE_DEPENDS "${PROJECT_SOURCE_DIR}/src/*.cpp")

add_executable(${PROJECT_NAME} ${SOURCES})

Link SFML (graphics, window, system)
target_link_libraries(${PROJECT_NAME} PRIVATE
 sfml-graphics
 sfml-window
 sfml-system
 sfml-audio
)

Optionally specify where to output the binary
set_target_properties(${PROJECT_NAME} PROPERTIES
 RUNTIME_OUTPUT_DIRECTORY ${PROJECT_SOURCE_DIR}/build
)
```

## c. Useful libraries (SFML):

*Install SFML 3.0.2:*

- Visit the GitHub release page for SFML 3.0.2: <https://github.com/SFML/SFML/releases/tag/3.0.2>.
- Scroll down to Source code (.zip) and click on it to download it.
- Make sure the .zip file will be downloaded to the **external** folder inside your project folder. (See the **folder layout** below.)
- Unzip the .zip file, and a new folder called “SFML-3.0.2” should appear inside your **external** folder.

- Delete the .zip file, and rename the folder to “SFML”.

## 4. Writing a starter Game/GUI program

In this tutorial, we will write a simple GUI program to display the frames per second (FPS) of the program.

Explanations for the code are provided in the code below as comments.

### a. Writing .cpp files:

- src/main.cpp:

```
#include "Game.hpp" // Import our Game class

int main() {
 // Create the Game object with a window size and title
 Game game(800, 600, "SFML Project Tutorial");
 // Start the main game loop
 game.run();
 return 0;
}
```

- src/Game.cpp:

```
#include "Game.hpp"
#include <iostream>
#include <iomanip>
#include <sstream>

// Constructor
Game::Game(unsigned int w, unsigned int h, const std::string& title)
: m_window(sf::VideoMode({w, h}), title), m_w(w), m_h(h), m_text(m_font),
circle(radius)
// Initialization list
{
 // Limit the frame rate from being too high
 m_window.setFramerateLimit(60);

 // Try to load a font from assets folder. If missing, print warning and
 continue.
 if (!m_font.openFromFile("assets/font.ttf")) {
 std::cerr << "Warning: failed to load assets/font.ttf -- text will not
display.\n";
 } else {
 // Configure the text object
 m_text.setFont(m_font); // Assign font
 m_text.setCharacterSize(18u); // Assign font size
 m_text.setPosition({8.f, 8.f}); // Set font position on the
window
 m_text.setString("SFML font loaded!"); // Set text to display
 }
}
```

```

// Fill the circle with red color
circle.setFillColor(sf::Color::Red);

// Set the origin to the center of the circle so setPosition coordinates
// represent the center, not the top-left corner.
circle.setOrigin({radius, radius});
}

void Game::run() {
 // Main game loop, executes until window is manually closed
 while (m_window.isOpen()) {
 processEvents(); // Keyboard & window actions
 float dt = m_clock.restart().asSeconds(); // Time since last frame
 update(dt); // Game logic (calculate FPS)
 render(); // Render (draw) everything
 }
}

void Game::processEvents() {
 // Poll all queued events
 while (auto ev = m_window.pollEvent()) {
 // If user clicks the close window button, close the window
 if (ev->is<sf::Event::Closed>()) m_window.close();
 // If the user presses the Escape key, close the window
 if (ev->is<sf::Event::KeyPressed>() &&
ev->getIf<sf::Event::KeyPressed>()->code == sf::Keyboard::Key::Escape)
 m_window.close();
 }
}

void Game::update(float dt) {
 // Simple FPS display using the loaded font (if available)
 if (!m_font.getInfo().family.empty()) {
 float fps = 1.f / dt;
 std::ostringstream ss;
 ss << "FPS: " << static_cast<int>(fps);
 m_text.setString(ss.str());
 }

 // Mouse logic
 // Get mouse position relative to the active window
 sf::Vector2i mousePos = sf::Mouse::getPosition(m_window);

 // Update circle position
 // We cast the integer mouse coordinates to float for the shape position
 circle.setPosition({static_cast<float>(mousePos.x),
static_cast<float>(mousePos.y)});
}

void Game::render() {

```

```

 // Clear screen with a dark background color
 m_window.clear(sf::Color{20, 24, 30});
 // Draw text object
 if (!m_font.getInfo().family.empty()) m_window.draw(m_text);
 // Draw the circle object
 m_window.draw(circle);
 // Display the frame
 m_window.display();
}

```

## b. Writing header (.hpp) files:

- include/Game.hpp:

```

#pragma once
#include <SFML/Graphics.hpp>
#include <string>

// The Game class manages:
// - Window creation
// - Event handling
// - Updating logic
// - Drawing text to the screen

class Game {
public:
 // Constructor: takes width, height, and window title
 Game(unsigned int w = 640, unsigned int h = 480, const std::string& title =
"Game");
 // Main loop (program runs here until window closes)
 void run();

private:
 // Handles keyboard + window events
 void processEvents();
 // Update game logic, dt = time elapsed between frames
 void update(float dt);
 // Draw all content to the window
 void render();

 // The game window
 sf::RenderWindow m_window;
 // Measures time between frames
 sf::Clock m_clock;

 // Font + HUD
 sf::Font m_font;
 sf::Text m_text;

 // Create the circle
 float radius = 30.f; // Circle radius

```

```
sf::CircleShape circle;

// Width and height of the window
unsigned int m_w, m_h;
};
```

### c. Adding assets

Get a font (.ttf) file, possibly downloaded from the Internet, and add it to the assets/ folder. Make sure to rename that .ttf file to “font.ttf”.

## 5. Building the project

- Open the terminal in your project root directory.
- Type the following into the terminal:

*macOS:*

```
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Release -S .. -B .
cmake --build .
```

*Windows:*

```
mkdir build
cd build
cmake -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release -S .. -B .
cmake --build .
```

**Notes:**

- + Building the project may take some time.
- + Every time you build your project, make sure you’re in your project’s root directory in the terminal before building.
- + The next time you build your project, you simply need to run the following commands in your terminal:

```
cd build
make
```

## 6. Running the program

Do not double-click the program to run it because it can result in errors when loading asset files. Do the following instead:

- Open the terminal in your project root directory.
- Type the following into the terminal:

(**Note:** Replace `MyProject` with your project name.)

```
On Linux/macOS:
cd build
```

```
./MyProject
On Windows (Visual Studio): run the generated solution or run the executable
from build\
cd build
.\MyProject.exe
```

**Note:** Every time you run your program, make sure you're in your project's root directory in the terminal before running.

## 7. Conclusions & Next steps

You now understand:

- How a multi-file C++ project is structured
- How SFML initializes a window
- How event polling works
- How delta-time is measured
- How fonts and text rendering, shape rendering work
- How the game loop processes and draws each frame

From now on, you can:

- Read more of SFML tutorials and documentation:  
<https://www.sfml-dev.org/tutorials/3.0/>
- Read the CMake tutorial:  
<https://cmake.org/cmake/help/latest/guide/tutorial/index.html>
- Try to create simple game projects like Tic-Tac-Toe, Pong, Minesweeper, etc.

# Task distribution and Responsibilities



**Class:** 25A02.

**Team ID:** 09.

**Members:**

- Nguyễn Phú Bảo Khang - 25125054.
- Nguyễn Phúc Khánh - 25125086.

**Responsibilities:**

- Nguyễn Phú Bảo Khang:
  - User Interface (UI.cpp, assets).
  - Testing and Documentation.
- Nguyễn Phúc Khánh:
  - Game Logic Implementation (Game.cpp, Board.cpp).
  - AI Development (AI.cpp).
  - Testing and Documentation.

**Collaboration Tools Used:** Git, GitHub, Facebook Messenger, Google Docs.

- *GitHub repository:* <https://github.com/khanhf-ng820/CS160-GoGame>.

# Testing

### ***1. Testing strategies:***

- The game is played for 30-minute intervals to check for bugs and errors in the user interface (clicking buttons, pressing keys, etc.) and in the game logic.
- We try to combine multiple actions together (passing and saving game files, placing a stone and resizing the window, etc.).
- Examples of bugs that have been found and fixed include:
  - Loading a new game still retains the old game's history
  - Saving the game should also save the number of stones captured by Black and White for the game state to be loaded more precisely
  - Fix the board overflowing from the window when the window is too small
  - Fix the main menu screen not being scaled properly when scaling the window,...

### ***2. Known issues and limitations:***

- Text might overflow the rectangular container that it's in.
- When the window is too small, the music file option boxes can be scrollable, but overflow by a small amount.
- The game window is sometimes not focused after running the program in the Terminal.

# Future Improvements

## ***Future Improvements:***

### ***1. UI/UX:***

- Sound notifications for stone captures, Ko threats.
- Allow separate settings for sound effects and background music.
- Different colors or themes for the stones.
- Allow the user to place stones using the keyboard.
- Display a list of moves using Go board coordinates.
- Add stone animations for a smoother user experience.
- Allow the settings to be saved when closing and reopening the app.

### ***2. Game:***

- Allow the user to select different rules and scoring systems to play.
- Include a captured stones counter and territory calculation counter.

# References

## References:

- Kitware, Inc. and Contributors. (2025). *CMake Tutorial*.  
<https://cmake.org/cmake/help/latest/guide/tutorial/index.html>
- Liên đoàn Cờ Việt Nam. (2025). *Luật chơi cờ vây cơ bản*.  
<https://vnchess.com.vn/luat-choi-co-vay-co-ban/>
- SFML Team. (2025). *SFML 3.0.2 documentation*.  
<https://www.sfml-dev.org/documentation/3.0.2/>
- SFML Team. (2025). *SFML 3.0 tutorial*.  
<https://www.sfml-dev.org/tutorials/3.0/>
- Wikipedia. (2025). *Rules of Go*. [https://en.wikipedia.org/wiki/Rules\\_of\\_Go](https://en.wikipedia.org/wiki/Rules_of_Go)
- AI Techniques for the Game of Go, Erik van der Werf, PHD Thesis, Universiteit Maastricht, 2005.

## Asset credits:

- *Music*:
  - C418. (2011). Dry Hands [Song]. On *Minecraft - Volume Alpha*.
  - C418. (2011). Minecraft [Song]. On *Minecraft - Volume Alpha*.
  - C418. (2011). Sweden [Song]. On *Minecraft - Volume Alpha*.
  - C418. (2011). Wet Hands [Song]. On *Minecraft - Volume Alpha*.
  - C418. (2013). Aria Math [Song]. On *Minecraft - Volume Beta*.
- *Sound effects*:
  - Pixabay. (2025). *Stone placing sound effect*. [Audio file]  
<https://pixabay.com/>.
- *Font*:
  - Matteson, S. (n.d.). *Open Sans* [Typeface]. Google Fonts.  
<https://fonts.google.com/specimen/Open+Sans>.